

Conceptual Session 1

JavaScript Foundations with a Problem-Solving Mindset

Document 1 of 2 — Concept Notes & Practice Problem Statements

Type A: solved live by the instructor | Type B: solved by students

Getting Started with JavaScript

What is JavaScript & Running JS in VSCode

JavaScript is the programming language that makes web pages interactive. Students will learn what JavaScript is used for and how to set up and run a .js file using VS Code and the Node.js runtime (or browser console).

Maths Needed for Programming

A quick refresher on the basic arithmetic and logical maths concepts (addition, subtraction, multiplication, division, remainder/modulus, comparison) that show up constantly while writing code.

What is a Variable? Five Things Needed to Declare One

A variable is a named container that holds a value. When you want to remember something in your program — like a student's name or their marks — you store it in a variable. Every time you need that value later, you use the variable name instead of writing the value again.



Think of a variable like a labelled box. You write a name on the outside of the box (variable name) and put something inside it (value). Whenever you need what is inside, you just call the box by its label.

Three keywords: const, let, var

Keyword	Can Change?	When to use
const	No — fixed value	Student name, subject names, fixed rules
let	Yes — can be updated	Running total, scores that change
var	Yes (old way)	Avoid using — use const or let instead

Syntax:

```
const studentName = 'Sakib';    // declare + assign in one step
let score = 85;

// Three things in one line:
// const → keyword
// studentName → variable name
// 'Sakib' → value
```

Rule: Variable names must start with a letter or underscore. No spaces. Use camelCase: studentName, totalMarks, isPassed.

Data Types: Numeric, String, Boolean, Undefined, Null, Array, Object

Not all values are the same kind. A student's name is text. Their marks are a number. Whether they passed is a yes or no. JavaScript uses different data types to represent these different kinds of information.



Think of data types like different compartments in a school bag. The pen slot holds only pens, the water bottle slot holds only bottles. In JavaScript, each type is a different kind of slot.

The 5 basic types

Type	Example Value	Represents
Number	85, 5.0, -10	Marks, GPA, age, price
String	"Sakib", "Dhaka"	Names, text, addresses
Boolean	true, false	Passed/failed, yes/no
Undefined	let x; (not assigned)	Variable declared but no value yet
Null	null	Intentionally empty value

Check the type with typeof:

```
const name = "Sakib";
const gpa = 5.0;
const passed = true;
const phone = null;
```

```
console.log(typeof name);    // "string"
console.log(typeof gpa);     // "number"
console.log(typeof passed);  // "boolean"
console.log(typeof phone);   // "object" ← known JS quirk, null is
                               not really object
```

Important: `typeof null` returns "object" — this is a well-known bug in JavaScript that was never fixed. `null` is not actually an object.

Keywords, Variable Names & Naming Conventions

JavaScript reserves certain words (like `let`, `const`, `function`) that cannot be used as variable names. Students will learn the rules for valid variable names and the `camelCase` convention used across the JavaScript community.

Fundamentals of JS Numbers

In JavaScript, all numbers (whole or decimal) belong to a single `Number` type. This section covers how numbers behave, common pitfalls (like floating-point precision), and useful `Number`-related checks.

Arithmetic Operators

Arithmetic operators let us do math with numbers in JavaScript — exactly the same as regular math, just written in code. We use these to calculate totals, averages, remainders, and powers.



Think of operators as the buttons on a calculator. `+` adds, `-` subtracts, `*` multiplies, `/` divides. The `%` button gives you the remainder after division — like when you share equally and have some left over.

Symb ol	Name	Example	Result
+	Addition	<code>72 + 85 + 90</code>	247
-	Subtraction	<code>100 - 15</code>	85
*	Multiplication	<code>5 * 3</code>	15
/	Division	<code>247 / 3</code>	82.33...
%	Remainder (Modulus)	<code>10 % 3</code>	1 (3 goes into 10 three times, leftover 1)
**	Power (Exponent)	<code>2 ** 10</code>	1024

Live example — Marks Calculator:

```
const bangla = 72;
const english = 85;
const math = 90;

const total = bangla + english + math; // 247
const average = total / 3; // 82.333...

console.log("Total:", total);
console.log("Average:", average.toFixed(2)); // 82.33
```

Mathematical Operation Shorthand (Advanced)

Shorthand assignment operators such as `+=`, `-=`, `*=`, `/=`, `++`, and `--` let you update a variable's value more concisely instead of writing the full expression each time.

Type A

1. Declare three variables to store a student's name, age, and pass/fail status using the correct data type for each, then print all three to the console.
2. Given the length and width of a rectangle, use arithmetic operators to calculate and print its area and perimeter.
3. Starting with a variable score = 50, use shorthand operators to first increase it by 10 and then multiply the result by 2. Print the final value.
4. Declare variables for a person's name, age, and height using the correct data types, then print them to the console.
5. Given two numbers, write code to calculate and print their sum, difference, product, and quotient using arithmetic operators.
6. Take a temperature value in Celsius and convert it to Fahrenheit using the formula $F = (C \times 9/5) + 32$.
7. Demonstrate shorthand operators (`+=`, `-=`, `*=`, `/=`) by updating a single variable's value step by step and printing the result after each step.
8. Write code that swaps the values of two variables without using a third (temporary) variable, using arithmetic shorthand.

Type B

1. Declare variables to store the price and quantity of an item, then calculate and print the total cost (price $\times$ quantity).
2. Take two variables holding numeric values and swap their values using a third (temporary) variable, then print both values after swapping.

3. Starting with lives = 5 and coins = 40, use shorthand operators to decrease lives by 1 and divide coins by 2. Print both final values.
4. Declare three variables to store a favorite movie's name, release year, and rating, using the correct data types for each.
5. Calculate the area and perimeter of a rectangle given its length and width.
6. Convert a given amount of money in Taka to US Dollars using a fixed conversion rate (e.g., 1 USD = 110 BDT).
7. Write a program to find the average of three numbers entered as variables.
8. Using shorthand operators, increase a variable named score by 10, then double it, then subtract 5. Print the final value

Making Decisions in Code

Comparing Variables & Comparison Operators

Comparison operators compare two values and always give us a Boolean result — either true or false. We use these to check conditions: Is this student's GPA high enough? Is this person old enough to apply?



Think of comparison operators like a weighing scale. You put two things on each side and the scale tells you which is heavier, or if they are equal. The result is always a simple yes (true) or no (false).

Operator	Meaning	Example	Result
>	Greater than	90 > 80	true
<	Less than	70 < 60	false
>=	Greater or equal	80 >= 80	true
<=	Less or equal	50 <= 40	false
===	Strictly equal	"A" === "A"	true
!==	Strictly not equal	5 !== 3	true

== vs ===: Always use === (strict equality) in JavaScript. == does type conversion and can give unexpected results: '5' == 5 is true. '5' === 5 is false. Always use ===.

Introduction to Conditionals

Conditionals let a program choose which block of code to run based on whether a condition is true or false, allowing programs to make decisions.

if / else Statement

The if statement runs a block of code only when its condition is true; the else block runs when it is false, giving the program exactly two possible paths.

Conditional Branching with if-else

A deeper look at how the program 'branches' — evaluating a condition once and following exactly one path through the code.

Logical Operators (&&, || and !)

Logical operators let us combine multiple conditions into one check. Instead of writing two separate if statements, we can ask: 'Is GPA high enough AND is the student old enough?' - all in one line.



Think of && like a chain. Every link must hold for the chain to be strong. Think of || like a light switch with two switches — if either switch is ON, the light turns on. ! is simply flipping a switch from on to off.

Operator	Name	Rule	Example
&&	AND	BOTH conditions must be true	<code>gpa >= 5.0 && age >= 18</code>
	OR	At least ONE must be true	<code>isStudent isWeekend</code>
!	NOT	Flips true to false, false to true	<code>!isPassed → true if failed</code>

Live examples:

```
const gpa = 5.0;
const age = 19;

// AND — both must be true
if (gpa >= 5.0 && age >= 18) {
  console.log("Eligible for admission");
}

// OR — at least one must be true
if (isStudent || isWeekend) {
  console.log("You get a discount");
}

// NOT — flips the boolean
const isPassed = false;
if (!isPassed) {
  console.log("Student needs to retake the exam");
}
```

Multi-Level if-else if-else

When there are more than two possible outcomes, else if lets you chain multiple conditions together and check them in order.

Nested if-else (Optional)

An if-else statement placed inside another if-else block, used when a decision depends on the outcome of a previous decision.

Ternary Operator (Advanced)

The ternary operator (condition ? valueIfTrue : valueIfFalse) is a compact, one-line shorthand for a simple if-else statement.

Logical NOT Operator (Advanced)

The NOT operator (!) reverses a Boolean value — turning true into false and false into true — and is useful for checking that a condition is not met.

Type A

1. Write a program that checks a given number and prints whether it is Positive, Negative, or Zero.
2. Using the logical AND operator, check whether a person is eligible to vote by verifying that their age is 18 or above AND they hold a valid ID (true/false).
3. Check whether a student passes or fails an exam based on marks (pass mark is 40) using a conditional statement.
4. Find the largest of three given numbers using a multi-level if-else if-else structure.
5. Using the ternary operator, write a one-line program that checks whether a given number is Even or Odd.

Type B

1. Write a program to check whether a given year is a leap year (use logical operators to combine the divisibility conditions).
2. Write a program that checks a student's marks and prints 'Pass' if the marks are 40 or above, otherwise 'Fail'.
3. Using multi-level if-else if-else, categorize a person's age into Child (0–12), Teen (13–19), Adult (20–59), or Senior (60+).
4. Check whether a person can apply for a driving license by verifying both age (18+) and having a valid ID (a boolean variable), using the logical AND operator.
5. Check whether a given character is a vowel or a consonant using a multi-level if-else structure.
6. Using the logical NOT operator, write a program that checks if a user is NOT logged in (isLoggedIn = false) and prints an appropriate message.
7. Using the logical NOT operator, write a program that checks a Boolean variable isLightOn and prints 'Switch is OFF' when the light is not on.

Repeating with Loops

What is a Loop? The for Loop

A loop repeats a block of code multiple times without rewriting it. The for loop is used when the number of repetitions is known in advance, using an initializer, a condition, and an increment/decrement.

Introduction to the while Loop

A while loop repeats a block of code as long as its condition remains true, and is typically used when the number of repetitions isn't known ahead of time.

Problem-Solving with while Loop

Applying the while loop to solve small real-world style problems, reinforcing how the loop condition and update step control execution.

Introduction to the for Loop

A closer, hands-on look at the for loop's three parts (initialization, condition, increment/decrement) and how they work together on each iteration.

Problem-Solving with for Loop

Applying the for loop to solve small real-world style problems to build fluency with iteration.

Different Ways to Use a Loop

Loops can count up, count down, skip values, or iterate over collections — this section explores those different usage patterns.

break and continue

The break keyword exits a loop immediately, while continue skips the current iteration and moves on to the next one.

Introduction to the do-while Loop (Optional)

A do-while loop runs its code block once before checking the condition, guaranteeing at least one execution even if the condition is initially false.

Difference Between Types of Loops

A comparison of for, while, and do-while loops — when to choose each one based on the problem at hand.

Type A

1. Using a for loop, print all numbers from 1 to 10.
2. Calculate the sum of the first N natural numbers using a while loop.

3. Print all even numbers between 1 and 50, using continue to skip odd numbers inside the loop.
4. Using a for loop, print all numbers between 1 and 50 that are divisible by both 3 and 5.
5. Write a program to find the sum of digits of a number using both a for loop and a while loop, and note the difference in structure.
6. Write a program that prints numbers from 1 to 20 but stops immediately when it reaches 15, using break.

Type B

1. Using a for loop, print all even numbers between 1 and 20.
2. Using a while loop, calculate the factorial of a given number.
3. Using a do-while loop, print numbers counting down from 10 to 1.
4. Print numbers from 10 down to 1 using a while loop.
5. Calculate the factorial of a given number using a for loop.
6. Print all odd numbers from 1 to 100 using a do-while loop.
7. Write a program that counts how many numbers between 1 and 100 are divisible by 7, using continue to skip the rest.

Working with Arrays

Storing Multiple Values Using Array

An array is a single variable that can hold multiple values in an ordered list, which is far more practical than creating a separate variable for each value.

Array Length, Index, Get and Set by Index

Each value in an array has a numbered position called its index (starting at 0). The `.length` property tells you how many items are in the array, and you can read or update any item using its index.

Adding/Removing Elements: `push`, `pop`, `shift`, `unshift`

`push()` adds an item to the end, `pop()` removes the last item, `unshift()` adds an item to the start, and `shift()` removes the first item of an array.

Basic Array Methods — `slice`, `splice`, `indexOf`, `includes`, `join`, `concat` & `isArray`

An overview of commonly used built-in array methods and how `Array.isArray()`, `slice`, `splice`, `indexOf`, `includes`, `join`, `concat` is used to check whether a given value is actually an array.

```
let nums = [10, 20, 30, 40, 50];
console.log(nums.slice(1, 3));    // [20, 30] (original unchanged)
nums.splice(1, 2, 99);           // removes 20,30 and inserts 99
console.log(nums);               // [10, 99, 40, 50]
console.log(nums.indexOf(40));    // 2
console.log(nums.includes(99));   // true
console.log(nums.join(" - "));    // "10 - 99 - 40 - 50"
let merged = [1, 2].concat([3, 4]); // [1, 2, 3, 4]
console.log(Array.isArray(nums)); // true
console.log(typeof nums);         // "object"
```

Array Traversal Using `for` and `while` Loop

Traversal means visiting every element in an array one by one, which is commonly done using a `for` loop or a `while` loop combined with the array's index and length.

Reversing an Array With/Without `reverse()`

An array's order can be reversed using the built-in `.reverse()` method, or manually by writing your own logic — useful for understanding what's happening under the hood.

Sorting an Array — Problems and Solutions

Sorting rearranges array elements into a specific order (like ascending or descending), which can be done using the built-in `.sort()` method or through manual comparison logic.

Type A

1. Create an array of 5 fruit names. Add a new fruit to the end using `push()`, then remove the last fruit using `pop()`. Print the array after each step.
2. Given an array of numbers, use a for loop to print each element along with its index.
3. Write a program to reverse a given array without using the built-in `reverse()` method.
4. Write a program that finds and prints the sum and average of all numbers in a given array.
5. Write a program that counts how many even numbers exist in a given array using a loop.
6. Given an array of numbers, write a program that finds and prints the second largest number without using `sort()`.
7. Given an array of 7 numbers, use `slice()` to copy out and print only the middle three elements, then confirm the original array is unchanged.
8. Given an array of names, use `splice()` to remove the 2nd and 3rd elements and insert two new names in their place at the same position. Print the array before and after.

Type B

1. Create an array of 5 numbers. Use `unshift()` to add a new number to the beginning, then use `shift()` to remove the first number. Print the array after each step.
2. Write a program that finds and prints the largest number in a given array using a loop.
3. Write a program that sorts a given array of numbers in ascending order.
4. Write a program that checks whether a given value exists inside an array using a loop, and prints true or false.
5. Write a program that removes duplicate values from a given array of numbers.
6. Write a program that finds the smallest number in a given array without using any built-in method.
7. Given an array of numbers, use `indexOf()` and `includes()` to check for a specific value and print its position (or -1) along with a true/false result.
8. Given two separate arrays of numbers, use `concat()` to merge them into one array, then use `join()` to print all values as a single comma-separated string.
9. Write a program that checks whether a given variable is an array using `Array.isArray()`, and prints an appropriate message for both an array input and a non-array input (e.g., a string).
10. Given an array of 6 numbers, use `splice()` to remove the first 2 elements without inserting anything, and print the removed elements as well as the remaining array.